COMMON SCHEMA MISTAKES TO AVOID

Even with Cedar's type safety, a poorly designed schema can lead to ongoing problems for your authorization model. Here are some pitfalls ACME's identity team learned to avoid:

Skipping action typing—Avoid using `principal: Any` or `resource: Any` in actions because they remove guardrails and can lead to accidental access grants. Without specific typing, any principal or resource can match, allowing new entity types to silently become eligible for the action. The solution involves several steps: start with the narrowest `appliesTo` possible, enumerate types instead of using `Any`, model separate actions for different cases, use tests to define scope precisely, and reserve `forbid` for real guardrails rather than fix loose typing issues.

Overloading attributes—Combining multiple concepts into one attribute (like `"department-role": "Eng-Manager"`) requires policies to parse strings or rely on fragile conventions. This makes conditions more difficult to read and maintain, increasing the risk of misinterpretation when data changes. Instead, break down complex data into separate typed attributes or related entities (`department: String`, `role: String`). This keeps policies simple, easier to test, and less prone to subtle logic bugs.

Inconsistent naming—Using different naming styles for entity types, attributes, or action IDs (such as using `"doc:view"` in one place and `"document-view"` in another) causes confusion and mismatched references in policies. The schema should be the place to establish and follow standard naming conventions: decide early on aspects like singular versus plural, casing, separators, and namespace prefixes, then apply them consistently. Clear naming makes onboarding new policy authors easier and reduces the chance of hard-to-spot typos.

Forgetting context—Excluding request-time attributes like device trust, current time, and geolocation from the schema forces developers to either hard-code them into the application or skip important checks altogether. Declaring context in the schema (such as `context.device.managed: Bool`) makes these attributes accessible everywhere policies are applied. It also ensures your test environment can simulate real-world scenarios,